

...ing is decidable. ... a given finite automaton. Hence d

**THEOREM 4.1**

ADFA is a decidable language.

195

**PROOF IDEA** We simply need to present a TM  $M$  that decides  $A_{DFA}$ .

$M =$  "On input  $\langle B, w \rangle$ , where  $B$  is a DFA and  $w$  is a string:

1. Simulate  $B$  on input  $w$ .
2. If the simulation ends in an accept state, *accept*. If it ends in a nonaccepting state, *reject*."

**PROOF** We mention just a few implementation details of this proof. For those of you familiar with writing programs in any standard programming language, imagine how you would write a program to carry out the simulation.

First, let's examine the input  $\langle B, w \rangle$ . It is a representation of a DFA  $B$  together with a string  $w$ . One reasonable representation of  $B$  is simply a list of its five components:  $Q, \Sigma, \delta, q_0$ , and  $F$ . When  $M$  receives its input,  $M$  first determines whether it properly represents a DFA  $B$  and a string  $w$ . If not,  $M$  rejects.

Then  $M$  carries out the simulation directly. It keeps track of  $B$ 's current state and  $B$ 's current position in the input  $w$  by writing this information down on its tape. Initially,  $B$ 's current state is  $q_0$  and  $B$ 's current input position is the leftmost symbol of  $w$ . The states and position are updated according to the specified transition function  $\delta$ . When  $M$  finishes processing the last symbol of  $w$ ,  $M$  accepts the input if  $B$  is in an accepting state;  $M$  rejects the input if  $B$  is in a nonaccepting state.

---

We can prove a similar theorem for nondeterministic finite automata. Let

$$A_{NFA} = \{ \langle B, w \rangle \mid B \text{ is an NFA that accepts input string } w \}.$$

## THEOREM 4.2

---

$A_{NFA}$  is a decidable language.

**PROOF** We present a TM  $N$  that decides  $A_{NFA}$ . We could design  $N$  to operate like  $M$ , simulating an NFA instead of a DFA. Instead, we'll do it differently to illustrate a new idea: Have  $N$  use  $M$  as a subroutine. Because  $M$  is designed to work with DFAs,  $N$  first converts the NFA it receives as input to a DFA before passing it to  $M$ .

$N =$  "On input  $\langle B, w \rangle$ , where  $B$  is an NFA and  $w$  is a string:

1. Convert NFA  $B$  to an equivalent DFA  $C$ , using the procedure for this conversion given in Theorem 1.39.
2. Run TM  $M$  from Theorem 4.1 on input  $\langle C, w \rangle$ .
3. If  $M$  accepts, *accept*; otherwise, *reject*."

Running TM  $M$  in stage 2 means incorporating  $M$  into the design of  $N$  as a subprocedure.



Similarly, we can determine whether a regular expression generates a given string. Let  $A_{\text{REX}} = \{\langle R, w \rangle \mid R \text{ is a regular expression that generates string } w\}$ .

### THEOREM 4.3

$A_{\text{REX}}$  is a decidable language.

**PROOF** The following TM  $P$  decides  $A_{\text{REX}}$ .

$P =$  "On input  $\langle R, w \rangle$ , where  $R$  is a regular expression and  $w$  is a string:

1. Convert regular expression  $R$  to an equivalent NFA  $A$  by using the procedure for this conversion given in Theorem 1.54.
2. Run TM  $N$  on input  $\langle A, w \rangle$ .
3. If  $N$  accepts, *accept*; if  $N$  rejects, *reject*."

Theorems 4.1, 4.2, and 4.3 illustrate that, for decidability purposes, it is equivalent to present the Turing machine with a DFA, an NFA, or a regular expression because the machine can convert one form of encoding to another.

Now we turn to a different kind of problem concerning finite automata: *emptiness testing* for the language of a finite automaton. In the preceding three theorems we had to determine whether a finite automaton accepts a particular string. In the next proof we must determine whether or not a finite automaton accepts any strings at all. Let

$$E_{\text{DFA}} = \{\langle A \rangle \mid A \text{ is a DFA and } L(A) = \emptyset\}.$$

### THEOREM 4.4

$E_{\text{DFA}}$  is a decidable language.

**PROOF** A DFA accepts some string iff reaching an accept state from the start state by traveling along the arrows of the DFA is possible. To test this condition, we can design a TM  $T$  that uses a marking algorithm similar to that used in Example 3.23.

$T =$  "On input  $\langle A \rangle$ , where  $A$  is a DFA:

1. Mark the start state of  $A$ .
2. Repeat until no new states get marked:
3. Mark any state that has a transition coming into it from any state that is already marked.
4. If no accept state is marked, *accept*; otherwise, *reject*."



The next theorem states that determining whether two DFAs recognize the same language is decidable. Let

$$EQ_{DFA} = \{\langle A, B \rangle \mid A \text{ and } B \text{ are DFAs and } L(A) = L(B)\}.$$

### THEOREM 4.5

$EQ_{DFA}$  is a decidable language.

**PROOF** To prove this theorem, we use Theorem 4.4. We construct a new DFA  $C$  from  $A$  and  $B$ , where  $C$  accepts only those strings that are accepted by either  $A$  or  $B$  but not by both. Thus, if  $A$  and  $B$  recognize the same language,  $C$  will accept nothing. The language of  $C$  is

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B)).$$

This expression is sometimes called the *symmetric difference* of  $L(A)$  and  $L(B)$  and is illustrated in the following figure. Here,  $\overline{L(A)}$  is the complement of  $L(A)$ . The symmetric difference is useful here because  $L(C) = \emptyset$  iff  $L(A) = L(B)$ . We can construct  $C$  from  $A$  and  $B$  with the constructions for proving the class of regular languages closed under complementation, union, and intersection. These constructions are algorithms that can be carried out by Turing machines. Once we have constructed  $C$ , we can use Theorem 4.4 to test whether  $L(C)$  is empty. If it is empty,  $L(A)$  and  $L(B)$  must be equal.

$F =$  "On input  $\langle A, B \rangle$ , where  $A$  and  $B$  are DFAs:

1. Construct DFA  $C$  as described.
2. Run TM  $T$  from Theorem 4.4 on input  $\langle C \rangle$ .
3. If  $T$  accepts, *accept*. If  $T$  rejects, *reject*."

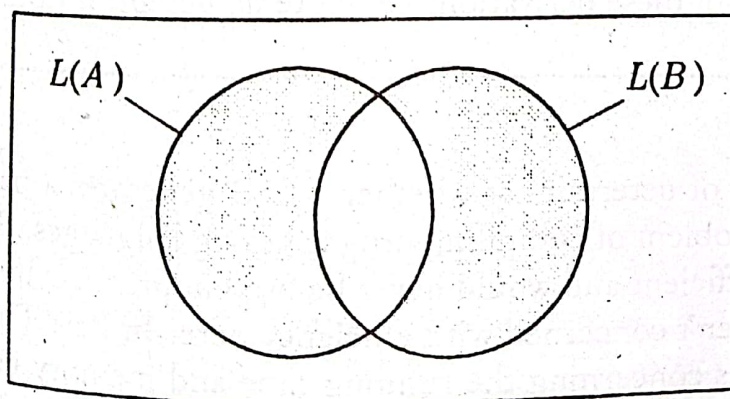


FIGURE 4.6

The symmetric difference of  $L(A)$  and  $L(B)$



## DECIDABLE PROBLEMS CONCERNING CONTEXT-FREE LANGUAGES

Here, we describe algorithms to determine whether a CFG generates a particular string and to determine whether the language of a CFG is empty. Let

$$A_{CFG} = \{ \langle G, w \rangle \mid G \text{ is a CFG that generates string } w \}.$$

### THEOREM 4.7

$A_{CFG}$  is a decidable language.

**PROOF IDEA** For CFG  $G$  and string  $w$ , we want to determine whether  $G$  generates  $w$ . One idea is to use  $G$  to go through all derivations to determine whether any is a derivation of  $w$ . This idea doesn't work, as infinitely many derivations may have to be tried. If  $G$  does not generate  $w$ , this algorithm would never halt. This idea gives a Turing machine that is a recognizer, but not a decider, for  $A_{CFG}$ .

To make this Turing machine into a decider, we need to ensure that the algorithm tries only finitely many derivations. In Problem 2.26 (page 157) we showed that if  $G$  were in Chomsky normal form, any derivation of  $w$  has  $2n - 1$  steps, where  $n$  is the length of  $w$ . In that case, checking only derivations with  $2n - 1$  steps to determine whether  $G$  generates  $w$  would be sufficient. Only finitely many such derivations exist. We can convert  $G$  to Chomsky normal form by using the procedure given in Section 2.1.

**PROOF** The TM  $S$  for  $A_{CFG}$  follows.

$S =$  "On input  $\langle G, w \rangle$ , where  $G$  is a CFG and  $w$  is a string:

1. Convert  $G$  to an equivalent grammar in Chomsky normal form.
2. List all derivations with  $2n - 1$  steps, where  $n$  is the length of  $w$ ; except if  $n = 0$ , then instead list all derivations with one step.
3. If any of these derivations generate  $w$ , *accept*; if not, *reject*."

The problem of determining whether a CFG generates a particular string is related to the problem of compiling programming languages. The algorithm in TM  $S$  is very inefficient and would never be used in practice, but it is easy to describe and we aren't concerned with efficiency here. In Part Three of this book, we address issues concerning the running time and memory use of algorithms. In the proof of Theorem 7.16, we describe a more efficient algorithm for recognizing general context-free languages. Even greater efficiency is possible for recognizing deterministic context-free languages.



Recall that we have given procedures for converting back and forth between CFGs and PDAs in Theorem 2.20. Hence everything we say about the decidability of problems concerning CFGs applies equally well to PDAs.

Let's turn now to the emptiness testing problem for the language of a CFG. As we did for DFAs, we can show that the problem of determining whether a CFG generates any strings at all is decidable. Let

$$E_{CFG} = \{ \langle G \rangle \mid G \text{ is a CFG and } L(G) = \emptyset \}.$$

#### THEOREM 4.8

$E_{CFG}$  is a decidable language.

**PROOF IDEA** To find an algorithm for this problem, we might attempt to use TM  $S$  from Theorem 4.7. It states that we can test whether a CFG generates some particular string  $w$ . To determine whether  $L(G) = \emptyset$ , the algorithm might try going through all possible  $w$ 's, one by one. But there are infinitely many  $w$ 's to try, so this method could end up running forever. We need to take a different approach.

In order to determine whether the language of a grammar is empty, we need to test whether the start variable can generate a string of terminals. The algorithm does so by solving a more general problem. It determines *for each variable* whether that variable is capable of generating a string of terminals. When the algorithm has determined that a variable can generate some string of terminals, the algorithm keeps track of this information by placing a mark on that variable.

First, the algorithm marks all the terminal symbols in the grammar. Then, it scans all the rules of the grammar. If it ever finds a rule that permits some variable to be replaced by some string of symbols, all of which are already marked, the algorithm knows that this variable can be marked, too. The algorithm continues in this way until it cannot mark any additional variables. The TM  $R$  implements this algorithm.

#### PROOF

$R =$  "On input  $\langle G \rangle$ , where  $G$  is a CFG:

1. Mark all terminal symbols in  $G$ .
2. Repeat until no new variables get marked:
3. Mark any variable  $A$  where  $G$  has a rule  $A \rightarrow U_1 U_2 \dots U_k$  and each symbol  $U_1, \dots, U_k$  has already been marked.
4. If the start variable is not marked, *accept*; otherwise, *reject*."



Next, we consider the problem of determining whether two context-free grammars generate the same language. Let

$$EQ_{CFG} = \{ \langle G, H \rangle \mid G \text{ and } H \text{ are CFGs and } L(G) = L(H) \}.$$

Theorem 4.5 gave an algorithm that decides the analogous language  $EQ_{DFA}$  for finite automata. We used the decision procedure for  $EQ_{DFA}$  to prove that  $EQ_{DFA}$  is decidable. Because  $EQ_{CFG}$  also is decidable, you might think that we can use a similar strategy to prove that  $EQ_{CFG}$  is decidable. But something is wrong with this idea! The class of context-free languages is *not* closed under complementation or intersection, as you proved in Exercise 2.2. In fact,  $EQ_{CFG}$  is not decidable. The technique for proving so is presented in Chapter 5.

Now we show that context-free languages are decidable by Turing machines.

#### THEOREM 4.9

Every context-free language is decidable.

**PROOF IDEA** Let  $A$  be a CFL. Our objective is to show that  $A$  is decidable. One (bad) idea is to convert a PDA for  $A$  directly into a TM. That isn't hard to do because simulating a stack with the TM's more versatile tape is easy. The PDA for  $A$  may be nondeterministic, but that seems okay because we can convert it into a nondeterministic TM and we know that any nondeterministic TM can be converted into an equivalent deterministic TM. Yet, there is a difficulty. Some branches of the PDA's computation may go on forever, reading and writing the stack without ever halting. The simulating TM then would also have some non-halting branches in its computation, and so the TM would not be a decider. A different idea is necessary. Instead, we prove this theorem with the TM  $S$  that we designed in Theorem 4.7 to decide  $A_{CFG}$ .

**PROOF** Let  $G$  be a CFG for  $A$  and design a TM  $M_G$  that decides  $A$ . We build a copy of  $G$  into  $M_G$ . It works as follows.

$M_G =$  "On input  $w$ :

1. Run TM  $S$  on input  $\langle G, w \rangle$ .
2. If this machine accepts, *accept*; if it rejects, *reject*."

Theorem 4.9 provides the final link in the relationship among the four main classes of languages that we have described so far: regular, context-free, decidable, and Turing-recognizable. Figure 4.10 depicts this relationship.



given input string. We call it  $A_{TM}$  by analogy with  $A_{DFA}$  and  $A_{CFG}$ . But, whereas  $A_{DFA}$  and  $A_{CFG}$  were decidable,  $A_{TM}$  is not. Let

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \}.$$

### THEOREM 4.11

$A_{TM}$  is undecidable.

Before we get to the proof, let's first observe that  $A_{TM}$  is Turing-recognizable. Thus, this theorem shows that recognizers *are* more powerful than deciders. Requiring a TM to halt on all inputs restricts the kinds of languages that it can recognize. The following Turing machine  $U$  recognizes  $A_{TM}$ .

$U$  = "On input  $\langle M, w \rangle$ , where  $M$  is a TM and  $w$  is a string:

1. Simulate  $M$  on input  $w$ .
2. If  $M$  ever enters its accept state, *accept*; if  $M$  ever enters its reject state, *reject*."

Note that this machine loops on input  $\langle M, w \rangle$  if  $M$  loops on  $w$ , which is why this machine does not decide  $A_{TM}$ . If the algorithm had some way to determine that  $M$  was not halting on  $w$ , it could *reject* in this case. However, an algorithm has no way to make this determination, as we shall see.

The Turing machine  $U$  is interesting in its own right. It is an example of the *universal Turing machine* first proposed by Alan Turing in 1936. This machine is called universal because it is capable of simulating any other Turing machine from the description of that machine. The universal Turing machine played an important early role in the development of stored-program computers.

## THE DIAGONALIZATION METHOD

The proof of the undecidability of  $A_{TM}$  uses a technique called *diagonalization*, discovered by mathematician Georg Cantor in 1873. Cantor was concerned with the problem of measuring the sizes of infinite sets. If we have two infinite sets, how can we tell whether one is larger than the other or whether they are of the same size? For finite sets, of course, answering these questions is easy. We simply count the elements in a finite set, and the resulting number is its size. But if we try to count the elements of an infinite set, we will never finish! So we can't use the counting method to determine the relative sizes of infinite sets.

For example, take the set of even integers and the set of all strings over  $\{0,1\}$ . Both sets are infinite and thus larger than any finite set, but is one of the two larger than the other? How can we compare their relative size?

Cantor proposed a rather nice solution to this problem. He observed that two finite sets have the same size if the elements of one set can be paired with the elements of the other set. This method compares the sizes without resorting to counting. We can extend this idea to infinite sets. Here it is more precisely.



Assume that we have sets  $A$  and  $B$  and a function  $f$  from  $A$  to  $B$ . Say that  $f$  is *one-to-one* if it never maps two different elements to the same place—that is, if  $f(a) \neq f(b)$  whenever  $a \neq b$ . Say that  $f$  is *onto* if it hits every element of  $B$ —that is, if for every  $b \in B$  there is an  $a \in A$  such that  $f(a) = b$ . Say that  $A$  and  $B$  are the *same size* if there is a one-to-one, onto function  $f: A \rightarrow B$ . A function that is both one-to-one and onto is called a *correspondence*. In a correspondence, every element of  $A$  maps to a unique element of  $B$  and each element of  $B$  has a unique element of  $A$  mapping to it. A correspondence is simply a way of pairing the elements of  $A$  with the elements of  $B$ .

Alternative common terminology for these types of functions is *injective* for one-to-one, *surjective* for onto, and *bijective* for one-to-one and onto.

#### EXAMPLE 4.13

Let  $\mathcal{N}$  be the set of natural numbers  $\{1, 2, 3, \dots\}$  and let  $\mathcal{E}$  be the set of even natural numbers  $\{2, 4, 6, \dots\}$ . Using Cantor's definition of size, we can see that  $\mathcal{N}$  and  $\mathcal{E}$  have the same size. The correspondence  $f$  mapping  $\mathcal{N}$  to  $\mathcal{E}$  is simply  $f(n) = 2n$ . We can visualize  $f$  more easily with the help of a table.

| $n$      | $f(n)$   |
|----------|----------|
| 1        | 2        |
| 2        | 4        |
| 3        | 6        |
| $\vdots$ | $\vdots$ |

Of course, this example seems bizarre. Intuitively,  $\mathcal{E}$  seems smaller than  $\mathcal{N}$  because  $\mathcal{E}$  is a proper subset of  $\mathcal{N}$ . But pairing each member of  $\mathcal{N}$  with its own member of  $\mathcal{E}$  is possible, so we declare these two sets to be the same size.

#### DEFINITION 4.14

A set  $A$  is *countable* if either it is finite or it has the same size as  $\mathcal{N}$ .

#### EXAMPLE 4.15

Now we turn to an even stranger example. If we let  $\mathcal{Q} = \{\frac{m}{n} \mid m, n \in \mathcal{N}\}$  be the set of positive rational numbers,  $\mathcal{Q}$  seems to be much larger than  $\mathcal{N}$ . Yet the two sets are the same size according to our definition. We give a correspondence with  $\mathcal{N}$  to show that  $\mathcal{Q}$  is countable. One easy way to do so is to list all the



| $n$      | $f(n)$      |
|----------|-------------|
| 1        | 3.14159...  |
| 2        | 55.55555... |
| 3        | 0.12345...  |
| 4        | 0.50000...  |
| $\vdots$ | $\vdots$    |

$$x = 0.4641 \dots$$

The preceding theorem has an important application to the theory of computation. It shows that some languages are not decidable or even Turing-recognizable, for the reason that there are uncountably many languages yet only countably many Turing machines. Because each Turing machine can recognize a single language and there are more languages than Turing machines, some languages are not recognized by any Turing machine. Such languages are not Turing-recognizable, as we state in the following corollary.

#### COROLLARY 4.18

Some languages are not Turing-recognizable.

**PROOF** To show that the set of all Turing machines is countable, we first observe that the set of all strings  $\Sigma^*$  is countable for any alphabet  $\Sigma$ . With only finitely many strings of each length, we may form a list of  $\Sigma^*$  by writing down all strings of length 0, length 1, length 2, and so on.

The set of all Turing machines is countable because each Turing machine  $M$  has an encoding into a string  $\langle M \rangle$ . If we simply omit those strings that are not legal encodings of Turing machines, we can obtain a list of all Turing machines.

To show that the set of all languages is uncountable, we first observe that the set of all infinite binary sequences is uncountable. An *infinite binary sequence* is an unending sequence of 0s and 1s. Let  $\mathcal{B}$  be the set of all infinite binary sequences. We can show that  $\mathcal{B}$  is uncountable by using a proof by diagonalization similar to the one we used in Theorem 4.17 to show that  $\mathcal{R}$  is uncountable.

Let  $\mathcal{L}$  be the set of all languages over alphabet  $\Sigma$ . We show that  $\mathcal{L}$  is uncountable by giving a correspondence with  $\mathcal{B}$ , thus showing that the two sets are the same size. Let  $\Sigma^* = \{s_1, s_2, s_3, \dots\}$ . Each language  $A \in \mathcal{L}$  has a unique sequence in  $\mathcal{B}$ . The  $i$ th bit of that sequence is a 1 if  $s_i \in A$  and is a 0 if  $s_i \notin A$ , which is called the *characteristic sequence* of  $A$ . For example, if  $A$  were the language of all strings starting with a 0 over the alphabet  $\{0, 1\}$ , its characteristic sequence  $\chi_A$  would be

$$\begin{array}{l} \Sigma^* = \{ \epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots \}; \\ A = \{ \quad 0, \quad \quad 00, 01, \quad \quad 000, 001, \dots \}; \\ \chi_A = \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad \dots \end{array}$$

The function  $f: \mathcal{L} \rightarrow \mathcal{B}$ , where  $f(A)$  equals the characteristic sequence of  $A$ , is one-to-one and onto, and hence is a correspondence. Therefore, as  $\mathcal{B}$  is uncountable,  $\mathcal{L}$  is uncountable as well.